

Don't Go Broke

Before You Trust the First Draft

Where This Booklet Fits:

This is booklet 4 of 5 in the Don't Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

- Field Note Opening
- 1. Why the First Draft Feels More Trustworthy Than It Is
- 2. The First Draft Review Session: A Walkthrough
- 3. The "Looks Done vs. Is Done" Test
- 4. The Screen and Data Review Protocol
- 5. Review by User Role
- 6. The Happy Path Trap
- 7. What Did AI Actually Decide for You?
- 8. Reviewing the Agent's Output: Files and Flows
- 9. The Interrogation Workflows
- 10. The First Draft Testing Worksheet
- 11. The Decision Guide: Accept, Revise, Revert, or Rebuild
- 12. Questions to Ask Before Continuing the Build
- 13. Before You Show It to Someone Else
- 14. The Safe Approval Protocol
- Field Note Closing

Before You Trust the First Draft

Field Note Opening

You asked an AI builder to create the first version of your product. A few minutes later, it returned something that looks real.

There are screens. There are buttons. There is a working navigation menu. You might even have a working demo you can click through.

That creates a dangerous feeling: *"It looks real, so it must be right."*

The moment you receive the first AI-generated draft is the moment you are most vulnerable to false confidence. The project looks much closer to the finish line than it actually is. It is incredibly tempting to keep the momentum going, to add more features immediately, or to show the prototype to an investor or a potential customer.

Stop.

This is not the point where trust should begin. This is the point where rigorous review must begin. The blueprint you made before building is now the standard you must review the first draft against.

The first draft is not proof that the product is correct. It is simply the first thing that must be questioned.

1. Why the First Draft Feels More Trustworthy Than It Is

When you build an app with AI, the results can be visually stunning within minutes. A polished interface, however, can hide weak thinking. A working demo can hide fragile logic. A confident AI response can hide a catastrophic architectural decision.

We are wired to believe that if something looks finished on the surface, the foundation must be solid. In traditional software development, reaching a working UI meant weeks of careful architectural planning, database design, and security modeling had already occurred.

With AI, the UI often comes first. The AI paints a beautiful exterior to mask a hollow or assumed foundation underneath.

Momentum creates false confidence. Do not mistake a slick button for a sound application.

2. The First Draft Review Session: A Walkthrough

Reviewing the first draft is not something you do passively while reading the AI's chat response. It is an active session. When the AI says, "I've built the first version," you must immediately switch from "Builder" mode to "Inspector" mode.

Here is the exact sequence of a safe review session:

1. **Pause Before Continuing:** Do not immediately ask the AI to add another feature. Do not click "Deploy." Do not send the link to a friend.
2. **Ask for the Change Receipt:** Force the AI to explicitly list what it built.
3. **Hunt for Assumptions:** Force the AI to confess what it guessed.
4. **Open the App as a User:** Step out of the codebase and click through the live demo.
5. **Test the Happy Path:** Verify that the core goal actually works.
6. **Break the Inputs:** Intentionally try to break the app.
7. **Test Alternate Roles:** Verify the app responds correctly to logged-out users, wrong users, and admins.
8. **Review the Files:** Look at the raw list of files the AI generated or modified.
9. **Decide on the Draft:** Make a final decision to Accept, Revise, Revert, or Rebuild.

The rest of this booklet provides the exact tools and worksheets you need to execute this session without needing an engineering degree.

3. The "Looks Done vs. Is Done" Test

If the app looks finished, you must actively prove that it isn't. The first step in reviewing a draft is learning to separate cosmetic completion from structural completion.

Use the "**Looks Done vs. Is Done**" Test to break the illusion of the first draft.

The Test

1. The "Fake Data" Check

- **Looks Done:** The dashboard is full of beautiful charts and user profiles.
- **Is Done:** When you create a *new* account, the charts actually pull real data from your inputs.
- *Warning:* AI loves to invent hardcoded sample data (sometimes called "mock data") to make a screen look finished. If the data doesn't change when you interact with the app, the app is not done.

2. The "Form Validation" Check

- **Looks Done:** There is a beautiful "Contact Us" form with a glowing submit button.
- **Is Done:** If you try to submit the form without typing an email address, it stops you and shows a red error message.
- *Warning:* AI often builds forms that look great but don't validate bad inputs. If you can submit a blank form and get a "Success!" message, the form is dangerously incomplete.

3. The "Security Illusion" Check

- **Looks Done:** There is a login screen with an email and password field.
 - **Is Done:** If you bypass the login screen and type the dashboard URL directly into your browser, the app kicks you out.
 - *Warning:* An AI will happily build a login screen that is completely disconnected from the actual data. Just because a door exists doesn't mean the walls are locked.
-

4. The Screen and Data Review Protocol

Once you establish that the app is structurally functioning, you must review exactly what it shows and how it behaves. Do not evaluate the app as a whole. Evaluate it screen by screen, and data point by data point.

The Screen Review Checklist

For every single generated screen, answer these questions:

- ☐ What is this screen supposed to do?
- ☐ What data does it pull from the database to show the user?
- ☐ What actions can the user take on this screen?
- ☐ What happens if the user views this screen but has no data yet?
- ☐ What happens if the user enters bad data or clicks the wrong button?
- ☐ What happens if the wrong user somehow accesses this screen?
- ☐ What data on this screen looks real but might be hardcoded fake data?
- ☐ What specific data point must be manually verified before I approve this screen?

The Data Behavior Review

A beautiful screen means nothing if the database behind it is broken. Use this beginner-safe guidance to review whether the draft actually saves, retrieves, protects, and updates data correctly.

- ☐ **The Persistence Check:** Does new data persist after a hard browser refresh?
 - ☐ **The Privacy Check:** Can one user see another user's private data?
 - ☐ **The Deletion Check:** When you click "Delete," does it actually delete the record from the database, or just hide it from the screen?
 - ☐ **The Edit Check:** Does editing a record update the right one, or does it accidentally overwrite something else?
 - ☐ **The Placeholder Check:** Are AI-generated placeholder records being mistaken for real user records?
 - ☐ **The Exposure Check:** Is private information (like an API key, an email address, or a password) displayed anywhere in the code or on the screen where it shouldn't be?
-

5. Review by User Role

AI builders often suffer from tunnel vision. If you ask an AI to build a "User Dashboard," it will build a dashboard that works perfectly for the "Ideal User." It will forget that other types of users exist.

You must teach yourself to test the draft not just as the creator, but as different kinds of users interacting with the system under different conditions.

The Role Testing Matrix

Test the application acting as:

- **The Logged-Out Visitor:** Can they accidentally see the dashboard without logging in?
- **The New User:** What happens when they log in for the first time? Does the app crash because they have no data, or does it show a friendly onboarding screen?
- **The Returning User:** Does the app remember them correctly?
- **The Wrong User:** If Alice logs in, can she view Bob's profile by changing the URL?
- **The User with Messy Data:** What happens if a user uploads a 10MB image instead of a 1MB image?
- **The Admin/Owner:** Do they have special privileges to manage the platform? Are those privileges secure?

Case Study: The Admin Access Mistake

A builder asked an AI to create a marketplace where users could buy and sell digital templates. The builder also asked for an "Admin Dashboard" to see all transactions and ban misbehaving users.

The AI built it rapidly. The marketplace looked great. The Admin Dashboard looked powerful. The builder tested the Admin Dashboard and saw all the expected buttons.

What Looked Finished: The Admin Dashboard was fully functional. **What the AI Assumed:** The AI assumed that simply

putting the Admin Dashboard on a hidden URL (`/admin-dashboard`) was enough security. It did not restrict access based on user accounts. **The Result:** A beginner could detect the issue by logging in as a regular user, typing `/admin-dashboard` into the URL bar, and discovering they had full power to ban other users.

What to Ask the AI: *"I noticed the admin dashboard is accessible by anyone who guesses the URL. Add strict role-based access control. Only users marked as 'Admin' in the database should be able to load this screen."*

In this case, the builder did not need to Rebuild the app. They just needed to Revise the security layer before continuing.

6. The Happy Path Trap

The "Happy Path" is the sequence of clicks where the user does exactly what they are supposed to do, and the app responds exactly how it is supposed to respond.

A smooth demo usually only shows the Happy Path.

If you ask an AI to build a shopping cart, it will build a path where a user adds an item, clicks checkout, and sees a success screen. The AI will mark the feature as "complete" because this single path works perfectly.

But what happens when the user clicks the "Checkout" button twice rapidly? What happens if they empty the cart and then try to check out? What happens if their internet disconnects right after they pay?

AI skips edge cases because you did not explicitly ask for them. A working demo is often incredibly fragile beneath the surface. It looks fine when you follow the exact script the AI expects, but it breaks the moment you step off that path.

Happy Path vs. Real Use: A Worked Example

Imagine you asked an AI to build a simple signup flow for a newsletter.

What Looks Good at First (The Happy Path): You type your email, click "Subscribe," and a beautiful green checkmark appears. The AI declares the feature finished.

What the AI Assumed: The AI assumed that users will only ever click the button once, that they will always type a valid email address, and that the database will never fail to save the record.

What Breaks When Tested (Real Use): You decide to test the edges. You type "not-an-email" and click Subscribe. It shows the green checkmark anyway. You leave the field blank and click Subscribe. Green checkmark. You click the button rapidly five times. The database saves five identical blank records.

What You Should Ask Next: Instead of moving on to building the email-sending feature, you must stop and address the foundation. You do not need to rebuild the whole app, but you must instruct the AI to revise the logic: *"The UI looks good, but it accepts blank emails, invalid formats, and double-clicks. Add strict form validation and disable the button while loading."*

The Edge-Case Breaker Sheet

To review the first draft, you must intentionally step off the Happy Path.

- ☐ **The Double-Click Test:** Click every primary button (Submit, Save, Pay) twice very fast. Does the app crash? Does it charge the user twice?
 - ☐ **The Empty State Test:** What happens when a user views a screen that has no data yet? (Does it show a helpful message, or does it crash trying to load an empty list?)
 - ☐ **The Refresh Test:** Navigate deep into the app, then refresh your browser page. Does it remember where you are, or does it crash and send you back to the home screen?
 - ☐ **The Back Button Test:** Fill out half a form, click the browser's "Back" button, and then click "Forward." Did it save your data, or did the app break?
-

7. What Did AI Actually Decide for You?

AI hates being stuck. If your prompt leaves out a detail, the AI will not stop and ask you for clarification. It will simply guess.

It will make assumptions about who can see what, how information is saved, what happens in unusual situations, and what the app does when you haven't told it what to do.

For example, you ask an AI to build a team dashboard. The AI might silently assume that anyone who creates an account should automatically have administrator access to view everyone else's financial data. It made a guess to keep the build moving.

You need to find these hidden decisions before you trust the code.

Hidden Assumption Case Study: The Fake Payment

A non-technical founder asked an AI agent to build a paid job board. They wrote a detailed prompt about the layout, the colors, and the pricing tiers.

The AI delivered a working prototype in ten minutes. The founder clicked "Post a Job," selected the \$50 tier, typed in a fake credit card, and the app showed a beautiful "Payment Successful! Your job is live!" screen. The job appeared on the homepage.

The founder was thrilled and immediately started showing the app to potential customers.

The Hidden Assumption: The AI had completely hallucinated the payment logic. Because the founder did not explicitly provide API keys for a real payment processor like Stripe, the AI wrote code that *simulated* a successful payment. It checked if a 16-digit number was entered, waited two seconds, and then just published the job to the database for free.

The founder could have detected this problem in two minutes without knowing how to read code, simply by interrogating the AI before trusting the draft.

The Hidden Assumption Finder

Use these questions to hunt down the AI's silent decisions:

- **Permissions:** Who is allowed to see this page? Did the AI assume everyone can see everything?
 - **Data Persistence:** When I refresh the page, where does the data come from? Is it actually saved in a real database, or just temporarily stored in the browser's local memory?
 - **Third-Party Services:** Did the AI sign me up for a paid service (like an email provider or a database host) in the code without asking me?
 - **Simulated Behavior:** Is this feature actually working, or did the AI hardcode a "success" screen to make it look finished?
 - **Scalability:** Did the AI write logic that works for 10 users but will crash if 1,000 users try to do it at the same time?
-

8. Reviewing the Agent's Output: Files and Flows

Review does not require an engineering degree, but it does require a methodical approach. Do not start by looking at the colors or the fonts. Start with the files and the flows.

File-by-File Review Rhythm

When an AI generates a draft, it outputs a list of files it created or modified. You must review this list critically using this rhythm:

1. **Look at Filenames First:** Do not read the code. Just read the names of the files the AI touched.
 2. **Compare Names to the Task:** If you asked it to build a pricing page, you should expect to see files like `Pricing.jsx` or `pricing.css`.
 3. **Flag Surprising Files:** If you asked for a pricing page, but the AI modified `UserAuthentication.ts` and `DatabaseConfig.js`, you have a massive red flag.
 4. **Ask Why:** Ask the AI explicitly: *"Why did you modify the DatabaseConfig file when I only asked for a pricing page?"*
 5. **Look for New Packages:** If the AI added a massive, complex third-party library just to format a date, challenge it.
 6. **Avoid Approving the Unexplainable:** Do not approve changes to files if you cannot explain, in plain English, why the AI touched them.
-

9. The Interrogation Workflows

Do not passively accept the AI's summary of its work. Force it to explain itself using structured interrogation.

The Change Receipt Interrogation

If the AI claims it "built the feature" or "fixed the bug," use this prompt to verify the true scope of the work:

"Output a detailed Change Receipt.

1. List every single file you modified or created.
2. For each file, explain in one plain-English sentence why it needed to be changed.
3. Did you modify any routing, authentication, styling, or dependency files? Answer 'Yes' or 'No' and explain.
4. Tell me what a human should manually verify before continuing."

The "What Did You Assume?" Interrogation

Copy and paste this into your chat to force the AI to confess its guesses:

"I am reviewing this draft.

1. List every technical, architectural, or user-experience assumption you made while generating this code that I did not explicitly request in my original prompt.
 2. Specifically list assumptions related to data security, user permissions, edge cases, and hardcoded values.
 3. List any fake, placeholder, or simulated behavior.
 4. List any security, privacy, data, payment, or account risks introduced by these assumptions."
-

10. The First Draft Testing Worksheet

Use this worksheet to track your review session. Do not move on to the next feature until this worksheet is complete.

Task Being Reviewed: _____

- ☐ **What the AI claims is done:** (e.g., "Login flow is complete.")
- ☐ **Happy Path Result:** Did it work when used perfectly? [Pass/Fail]
- ☐ **Broken-Input Result:** What happened when you entered bad data? [Handled/Crashed]
- ☐ **Empty-State Result:** What happened when there was no data? [Handled/Crashed]
- ☐ **Refresh Result:** What happened when you refreshed the page mid-flow? [Handled/Crashed]
- ☐ **Security Result:** Can the wrong user see private data? [Yes/No]
- ☐ **Assumptions Found:**

- ☐ **Files Changed Unexpectedly:**

Final Decision (Circle One): Accept / Revise / Revert /
Rebuild

11. The Decision Guide: Accept, Revise, Revert, or Rebuild

At the end of your review session, you must make a decision about what to do with the first draft. You do not have to keep everything the AI generated. You are allowed to throw parts of the draft away.

Use this decision guide to determine your next move:

1. ACCEPT

When to use it: The draft does exactly what you asked. You used the "Looks Done vs. Is Done" test, and it passed. The AI surfaced its assumptions, and you agree with them. **Action:** Commit the code. Consider this feature a stable foundation. You may now move on to the next feature.

2. REVISE

When to use it: The core logic is sound, but the AI missed a few edge cases. The empty states are missing, or the form doesn't validate inputs correctly. **Action:** Do not write a massive new prompt. Keep the current code, but issue narrow, specific

instructions to fix the gaps. (e.g., "The login logic is good, but add error validation for missing email addresses.")

3. REVERT

When to use it: You asked the AI to fix a small UI bug, but the Change Receipt shows it also rewrote your routing logic, updated three dependencies, and broke the navigation menu. **Action:** Throw the new changes away immediately. Use Git to revert to your last known good save point. Never ask the AI to "fix what it just broke." Restart the task with a stricter, narrower prompt.

4. REBUILD

When to use it: The draft is a mess of assumptions, hardcoded data, and over-engineered architecture. The UI looks nice, but the foundation is completely wrong for your actual goals. **Action:** Delete the entire draft. Do not try to patch a broken foundation. It is infinitely faster to delete bad code and prompt the AI again with clearer, more specific instructions than it is to untangle a hallucinated architecture.

12. Questions to Ask Before Continuing the Build

Before you write the next prompt or add the next feature, you must definitively answer these questions about the draft you just reviewed. Do not proceed until you have clear answers.

- ☐ What is safe to keep?
 - ☐ What is still fake?
 - ☐ What is assumed?
 - ☐ What is untested?
 - ☐ What is risky?
 - ☐ What must be rebuilt before adding features?
 - ☐ What should be committed to version control right now?
-

13. Before You Show It to Someone Else

There is a strong temptation to share the first draft immediately. You want feedback. You want validation. You want to show your investors or co-founders how fast you are moving.

But showing an unverified draft to collaborators or early users is incredibly dangerous.

If they see a polished UI, they will assume the app is almost done. They will start asking for tweaks to the button colors or the fonts rather than questioning the core logic.

Worse, if you show a demo that relies on hardcoded fake data or simulated payments, you are setting a false expectation of progress. This locks in bad assumptions and creates immense, artificial pressure on you to launch an app that isn't actually built yet.

Only share what you can explain. Sharing with others is incredibly useful, but only *after* you understand the core logic well enough to explain exactly what they are looking at and where the rough edges are. Make sure you understand what the draft is actually doing before handing it off.

14. The Safe Approval Protocol

Before you declare the first draft complete and move on to building the next major feature, execute this final protocol.

- ☐ **Review the Receipt:** I have read the Change Receipt and understand why every modified file was touched.
- ☐ **Test the Flow:** I have stepped off the Happy Path and tested broken inputs and empty states.
- ☐ **Test the Roles:** I have viewed the app as a logged-out visitor, a new user, and an admin.
- ☐ **Verify the Reality:** I have confirmed there is no fake, simulated, or placeholder behavior hiding behind the UI.
- ☐ **Approve the Assumptions:** I have interrogated the AI for its hidden guesses regarding permissions and data, and I approve them.
- ☐ **Save the Notes:** I have saved the results of my Testing Worksheet.
- ☐ **Explain it Simply:** If someone asked how this core logic works, I could explain it to them in plain English.

If you can check every box, the draft is "reviewed enough to continue."

Field Note Closing

Reviewing the first draft is not a delay from building; it is a critical part of building.

By taking the time to question the output, surface the assumptions, break the happy path, and verify the logic, you transform a fragile, generated draft into a trustworthy foundation.

Build with your eyes open. Once the foundation is reviewed and secure, you can safely move on to iteration, refinement, and eventually, the responsibilities of a live launch.

The Next Step

You are ready for the next phase. Proceed to booklet: **After Launch, the Meter Still Runs.**